Итог

Статус: Не готово к сдаче.

По содержанию реализация выглядит сильной и методически осмысленной: выбран реальный почасовой ряд AEP, предусмотрены очистка временной шкалы, анализ сезонностей, хронологическое разбиение 70/15/15, train-only нормализация, скользящие окна 168→1, сравнение LSTM и GRU, подбор конфигурации только по validation, две baseline-модели, метрики MAE/RMSE/R^2/sMAPE, анализ остатков и прогноз следующего часа. Однако файл practical_work_3_time_series_forecasting(2).ipynb сохранён полностью без результатов выполнения: у всех 24 кодовых ячеек execution_count=None, а outputs отсутствуют. Поэтому невозможно подтвердить фактическую загрузку Kaggle-датасета, результаты EDA, обучение LSTM/GRU, выбранную конфигурацию, test-метрики, графики и финальный прогноз. Дополнительно обнаружены несколько воспроизводимых методических рисков: интерполяция выполняется до временного split, scaler-объекты не сохраняются вместе с checkpoint, а поисковые эксперименты запускаются с разными seed и без сброса генератора shuffle. После полного выполнения notebook и небольших правок работа, вероятно, будет готова к сдаче.

Матрица соответствия

№	Источник	Требование	Статус	Доказательство	Замечание
1	Постановка	Прогнозировать AEP на 1 час вперёд по 168 предыдущим часам.	частично	Ячейки 2, 5, 21: LOOKBACK=168, HORIZON=1, WindowDataset.	Пайплайн есть, фактический прогноз не выполнен.
2	Постановка	Использовать реальный Hourly Energy Consumption / AEP_hourly.csv.	частично	Ячейки 6-8: KaggleHub, robikscube/hourly-energy-consumption, AEP_hourly.csv.	Код загрузки есть, фактическая загрузка не подтверждена.
3	Качество данных	Проверить типы, пропуски, статистики и дубликаты.	выполнено	Ячейка 8: quality_before, describe, min/max даты, дубликаты, пропуски.	Проверки реализованы кодом.
4	Подготовка данных	Сделать временную шкалу регулярной и обработать дубликаты.	выполнено	Ячейка 10: groupby Datetime, mean, sort, reindex с freq=1h.	Логика обработки присутствует.
5	Подготовка данных	Восстановить пропущенные часы.	частично	Ячейка 10: time interpolation, limit_direction=both.	Код есть, число восстановленных часов неизвестно без запуска.
6	EDA	Показать весь ряд, первый месяц и распределение нагрузки.	частично	Ячейка 12: три графика и сохранение eda_energy.png.	Графики не сохранены в outputs.
7	EDA	Исследовать суточную, недельную и годовую сезонность.	частично	Ячейка 13: средние по часу, дню недели и месяцу.	Код есть, фактические графики отсутствуют.
8	EDA	Исследовать автокорреляцию и обосновать окно.	частично	Ячейки 14-15: лаги до 336 ч, метки 24/168/336, вывод про 168 ч.	Вывод написан заранее; график выполнения отсутствует.
9	Признаки	Добавить циклические календарные признаки.	выполнено	Ячейка 17: sin/cos часа, дня недели и дня года.	Сформировано 6 календарных признаков плюс AEP_MW.
10	Split	Использовать хронологическое разбиение 70/15/15.	выполнено	Ячейка 18: train_end, val_end и таблица split.	Случайный split временного ряда не применяется.
11	Leakage	Обучать StandardScaler только на train.	выполнено	Ячейка 20: feature_scaler.fit и target_scaler.fit только на [:train_end].	Scaler leakage для масштабирования предотвращён.
12	Окна	Сформировать sliding windows 168×7 → 1.	выполнено	Ячейка 21: WindowDataset; assert формы (168, len(FEATURES)).	Статическая синтетическая проверка формы прошла.
13	Окна	Не допустить пересечения target между split.	выполнено	Ячейка 21: assert train < val < test по target_indices.	Границы целей разделены.
14	DataLoader	Настроить train shuffle и детерминированность.	выполнено	Ячейка 22: shuffle=True для train, фиксированный Generator seed.	Базовая воспроизводимость предусмотрена.
15	Модели	Реализовать LSTM и GRU.	выполнено	Ячейка 24: RecurrentRegressor выбирает nn.LSTM или nn.GRU.	Обе архитектуры реализованы одним классом.

№	Источник	Требование	Статус	Доказательство	Замечание
16	Обучение	Использовать устойчивый loss и оптимизатор.	выполнено	Ячейка 25: HuberLoss, AdamW.	Подход подходит для regression с выбросами.
17	Обучение	Использовать gradient clipping.	выполнено	Ячейка 24: clip_grad_norm_ max_norm=1.0.	Снижает риск взрыва градиентов.
18	Обучение	Использовать scheduler и early stopping.	выполнено	Ячейка 25: ReduceLROnPlateau и patience-based stopping.	Лучшая state_dict восстанавливается.
19	Подбор	Сравнить несколько LSTM/GRU конфигураций.	частично	Ячейки 27-28: 3 trial, SEARCH_EPOCHS=6, SEARCH_STRIDE=3.	Код есть, таблица search_df не выполнена.
20	Подбор	Выбрать конфигурацию только по validation.	выполнено	Ячейка 28: sort_values(best_val_loss), test не участвует.	Принцип выбора корректный.
21	Обучение	Заново обучить лучшую конфигурацию на полном train.	частично	Ячейка 30: train_model(best_config, train_loader, val_loader).	Фактическое обучение не подтверждено.
22	Checkpoint	Сохранить лучший checkpoint.	частично	Ячейки 25 и 30: best_state сохраняется в best_energy_rnn.pt.	Код есть, файл в приложении не подтверждён.
23	Мониторинг	Построить train/validation loss и MAE.	частично	Ячейка 31: два графика, best_epoch, training_history.png.	Outputs отсутствуют.
24	Test	Использовать test только для финальной оценки.	частично	Ячейки 32-33: test evaluation выполняется после model selection.	Принцип верный, фактические test-результаты отсутствуют.
25	Baseline	Добавить persistence baseline.	выполнено	Ячейка 33: значение предыдущего часа.	Корректный простой baseline.
26	Baseline	Добавить weekly seasonal baseline.	выполнено	Ячейка 33: значение 168 часов назад.	Содержательный baseline для недельной сезонности.
27	Метрики	Рассчитать MAE, RMSE, R^2 и sMAPE.	выполнено	Ячейка 33: regression_metrics и smape.	Синтетическая проверка метрик прошла.
28	Результаты	Сравнить RNN с baseline.	частично	Ячейка 33: test_results и изменение MAE относительно лучшего baseline.	Фактические значения отсутствуют.
29	Визуализация	Показать факт, RNN и weekly baseline на test.	частично	Ячейка 34: первые 14 суток test.	График forecast_vs_actual не сохранён в notebook output.
30	Ошибки	Проанализировать residuals и худшие ошибки.	частично	Ячейки 35-36: histogram/KDE, scatter, top-10 absolute errors.	Анализ нельзя подтвердить без запуска.
31	Inference	Сделать прогноз следующего часа.	частично	Ячейки 37-38: predict_next_hour и финальная точка на графике.	Реальное значение прогноза отсутствует.
32	Отчёт	Сформулировать итоговые выводы по результатам.	не выполнено	Notebook заканчивается графиком прогноза и не содержит отдельного финального вывода.	Нет итогового текста с выбранной моделью и метриками.
33	Воспроизводимость	Фиксировать случайность.	выполнено	Ячейка 4: random, NumPy, PyTorch, cuDNN deterministic.	Базовая детерминированность предусмотрена.
34	Безопасность	Не хранить секреты.	выполнено	Статический поиск токенов/паролей/API-ключей ничего не выявил.	Очевидных секретов нет.
35	Доказательства	Предоставить фактические результаты выполнения.	не выполнено	24/24 code cells имеют execution_count=None; outputs=0.	Главный блокер для приёмки.
36	Внешние файлы	Проверить README, requirements, tests и репозиторий.	не проверено	На аудит передан только notebook.	Отсутствие внешних файлов не считается ошибкой этого аудита.

Найденные проблемы

AUD-001 - Critical

Источник: Качество / доказательства выполнения

Описание: Notebook сохранён без единого результата выполнения. Нельзя подтвердить загрузку данных, EDA, подбор LSTM/GRU, обучение, checkpoint, test-метрики, residual analysis и прогноз следующего часа.

Как воспроизвести или проверить: Открыть practical_work_3_time_series_forecasting(2).ipynb и проверить execution_count и outputs всех code cells.

Ожидаемый и фактический результат: Ожидается выполненный notebook с таблицами, логами, графиками и числовыми метриками. Фактически 24 из 24 code cells имеют execution_count=None, общее число outputs равно 0.

Рекомендуемое исправление: Выполнить Runtime -> Run all в Colab/локальном Jupyter, дождаться завершения всех стадий и сохранить notebook вместе с outputs.

AUD-002 - Major

Источник: Отчёт / выводы

Описание: В notebook отсутствует финальный раздел с выводом. Работа заканчивается визуализацией прогноза следующего часа, но не суммирует, какая архитектура победила и насколько она лучше baseline.

Как воспроизвести или проверить: Просмотреть последние ячейки notebook после раздела «12. Анализ ошибок».

Ожидаемый и фактический результат: Ожидается итог, основанный на фактических MAE, RMSE, R^2, sMAPE и сравнении baseline. Фактически отдельного раздела «Вывод» нет.

Рекомендуемое исправление: Добавить финальный вывод: выбранная LSTM/GRU конфигурация, test-метрики, сравнение с лучшим baseline, наблюдения по residuals, ограничения и возможные улучшения.

AUD-003 - Major

Источник: Методология / утечка данных

Описание: Восстановление отсутствующих часов выполняется до хронологического split через time interpolation с limit_direction="both". В граничном случае пропуск в train может быть заполнен с использованием будущего значения из validation, а пропуск в validation - из test.

Как воспроизвести или проверить: Сравнить порядок ячеек: очистка и интерполяция выполняются в ячейке 10, а train/validation/test границы создаются только в ячейке 18.

Ожидаемый и фактический результат: Ожидается, что подготовка данных не использует будущую часть ряда для заполнения более раннего split. Текущий код потенциально допускает такое использование.

Рекомендуемое исправление: Либо выполнять imputation отдельно внутри каждого split, либо использовать причинный метод (например, forward fill), либо явно проверить и показать, что около границ split интерполированных точек нет.

AUD-004 - Major

Источник: Воспроизводимость / inference

Описание: Checkpoint сохраняет веса, config, features, lookback и horizon, но не сохраняет feature_scaler и target_scaler. Функция predict_next_hour зависит от scaler-объектов, оставшихся в памяти текущей сессии.

Как воспроизвести или проверить: Проверить содержимое torch.save в train_model и код predict_next_hour.

Ожидаемый и фактический результат: Ожидается возможность восстановить inference после перезапуска runtime из сохранённых артефактов. Фактически одного best_energy_rnn.pt недостаточно для точной нормализации.

Рекомендуемое исправление: Сохранить оба scaler через joblib либо включить их параметры в отдельный artifact; рядом с checkpoint сохранить metadata и пример кода загрузки.

AUD-005 - Major

Источник: Качество сравнения гиперпараметров

Описание: Три trial запускаются с разными seed (SEED + trial_id), а один и тот же search_loader с shuffle generator переиспользуется последовательно. Поэтому ranking конфигураций частично смешан с различиями случайной инициализации и порядка batch.

Как воспроизвести или проверить: Проверить ячейки 22 и 28: search_loader создаётся один раз; train_model вызывается с seed=SEED+trial_id.

Ожидаемый и фактический результат: Для более честного сравнения желательно одинаковое случайное условие для всех конфигураций либо несколько повторов каждого trial. Сейчас одиночный ranking может зависеть от seed.

Рекомендуемое исправление: Использовать один seed и пересоздавать DataLoader/Generator для каждого trial или оценивать каждую конфигурацию на нескольких seed и усреднять validation score.

AUD-006 - Minor

Источник: Портируемость

Описание: Каталог артефактов жёстко задан как /content/lab3_artifacts. Это удобно для Colab, но локальный Jupyter может не иметь каталога /content или прав на его создание.

Как воспроизвести или проверить: Проверить ARTIFACT_DIR в ячейке 5.

Ожидаемый и фактический результат: Ожидается переносимый notebook для Colab и локальной среды. Фактически путь привязан к Colab.

Рекомендуемое исправление: Определять среду выполнения и использовать /content только в Colab, а локально - Path.cwd()/lab3_artifacts или каталог проекта.

AUD-007 - Minor

Источник: Воспроизводимость окружения

Описание: Notebook импортирует kagglehub, joblib, scikit-learn, torch и другие пакеты, но внутри notebook нет установки/проверки версий зависимостей.

Как воспроизвести или проверить: Запустить notebook в новой среде без заранее подготовленного requirements.

Ожидаемый и фактический результат: Ожидается понятная процедура воспроизводимого запуска. Фактически успешность зависит от того, какие пакеты уже установлены.

Рекомендуемое исправление: Добавить компактную установочную ячейку для Colab либо ссылку на requirements.txt и печать версий ключевых библиотек.

AUD-008 - Minor

Источник: Строгий протокол оценки

Описание: Автокорреляция и визуальный EDA выполняются на всём ряду до выделения test, а затем используются для обоснования LOOKBACK=168. Это слабая форма использования информации о будущей части ряда при проектировании модели.

Как воспроизвести или проверить: Проверить порядок ячеек 12-15 и 18.

Ожидаемый и фактический результат: Для максимально строгого benchmark решения о feature engineering и lookback должны основываться на train. Текущий подход допустим для учебного EDA, но менее строг для независимой test-оценки.

Рекомендуемое исправление: При строгой защите вычислять диагностические статистики для выбора параметров на train и использовать полный ряд только для иллюстративных графиков после фиксации методологии.

Непроверяемые пункты

- Фактическая загрузка AEP_hourly.csv через KaggleHub - нужен выполненный вывод загрузочной ячейки.
- Реальное количество исходных дубликатов, пропусков и восстановленных часов - нужны outputs quality/cleaning.
- Фактические суточные, недельные и годовые зависимости - нужны сохранённые графики EDA.
- Значения autocorrelation на лагах 24, 168 и 336 часов - нужен выполненный график.
- Результаты трёх hyperparameter trial - нужна таблица search_df.
- Выбранная архитектура LSTM или GRU и её best validation loss - нужен фактический запуск.
- Успешное завершение финального обучения без ошибок памяти/времени - нужны логи эпох.
- Наличие и содержимое best_energy_rnn.pt - нужен артефакт или выполненный вывод.
- Фактические MAE, RMSE, R^2 и sMAPE на test - нужна test_results.
- Факт превосходства или проигрыша RNN относительно persistence/weekly baseline - нужны реальные метрики.
- Распределение residuals и 10 худших часов - нужны выполненные таблицы/графики.
- Численное значение прогноза следующего часа - нужен выполненный predict_next_hour.

Лишняя / расширенная реализация

- Агрегация повторных временных меток средним и восстановление регулярного hourly index.
- Шесть циклических calendar features через sin/cos.
- Отдельный feature scaler и target scaler, обучаемые только на train.
- Два содержательных baseline: persistence и weekly seasonal.
- Huber loss, AdamW, gradient clipping, ReduceLROnPlateau и early stopping.
- Автоматический выбор лучшей trial-конфигурации только по validation.
- Residual analysis и таблица 10 крупнейших абсолютных ошибок.
- Отдельная функция прогноза следующего часа и его визуализация.

Эти элементы не выглядят избыточными в отрицательном смысле: они повышают качество учебного эксперимента и делают сравнение модели с простыми временными baseline содержательным.

Результаты проверок

Проверка	Результат
Структура файла	44 138 байт; SHA256 13F0A7BBB4639E43BEDA2327AFF4B910F2598B69527655D9F19611E6F3E026FF; nbformat 4.5.
Ячейки	38 ячеек: 14 Markdown и 24 code. Выполненных code cells: 0. Outputs: 0.
nbformat.validate	Успешно. Структура Jupyter Notebook корректна.
AST-разбор Python	Все 24 code cells успешно разобраны; SyntaxError не обнаружены.
Поиск секретов	Очевидные API keys, passwords, GitHub tokens и подобные секреты не обнаружены.
Calendar features	Синтетическая проверка: формируется 7 признаков, значения конечны.
WindowDataset	Синтетическая проверка: первый объект имеет форму (168, 7), target индексируется корректно.
LSTM/GRU forward	Синтетическая CPU-проверка: обе модели выдают один prediction на sample; shape (4,) для batch=4.
Метрики	Синтетическая проверка MAE/RMSE/R^2/sMAPE прошла, конечные значения получены.
Полное обучение	Не выполнялось: notebook не содержит сохранённых результатов, сетевой Kaggle download и training намеренно не запускались в статическом аудите.

Подготовка к защите

Сценарий демонстрации работы по шагам

- Открыть notebook и показать Python/PyTorch device; при наличии использовать GPU.
- Показать загрузку AEP_hourly.csv и исходные проверки качества данных.
- Показать очистку временной шкалы, количество дубликатов и восстановленных часов.
- Продемонстрировать весь ряд, первый месяц и распределение энергопотребления.
- Показать сезонность по часу, дню недели и месяцу, затем autocorrelation с лагами 24/168/336.
- Объяснить выбор LOOKBACK=168 и HORIZON=1.
- Показать calendar features и хронологический split 70/15/15.
- Показать, что StandardScaler fit выполняется только на train.
- Показать форму одного окна (168, 7) и отсутствие пересечения target_indices split.
- Показать архитектуру RecurrentRegressor и объяснить различия LSTM/GRU.
- Показать таблицу трёх trial и объяснить выбор лучшей конфигурации по validation loss.

- Показать training history и best epoch финальной модели.
- Открыть test_results: persistence, weekly baseline и лучшая RNN по MAE/RMSE/R^2/sMAPE.
- Показать график факт vs RNN vs weekly baseline, затем residual analysis и top-10 ошибок.
- Выполнить predict_next_hour и показать итоговую прогнозную точку.
- Закончить защиту итоговым выводом: что победило, насколько лучше/хуже baseline и какие ограничения остаются.

Обязательные экраны, команды и тестовые данные

- Таблица data quality до очистки.
- Период данных и число восстановленных часов.
- EDA и seasonality plots.
- Autocorrelation с лагом 168.
- Таблица train/validation/test с датами.
- Средние и стандартные отклонения train после scaling.
- Shape одного WindowDataset sample.
- Таблица hyperparameter search.
- Training/validation loss и MAE.
- Test comparison table с четырьмя метриками.
- Forecast vs actual на двух неделях test.
- Residual plots и top-10 worst timestamps.
- Next-hour prediction.

Доказательства выполнения ключевых требований

- Полностью выполненный notebook с сохранёнными outputs.
- Логи подбора LSTM/GRU и финального обучения.
- Таблица validation search и test comparison.
- Checkpoint лучшей модели и scaler artifacts.
- Сохранённые EDA/training/forecast/residual plots.
- Финальный вывод с реальными метриками.

10 вероятных вопросов преподавателя и краткие ответы

1. Почему выбран AEP?
Это реальный многолетний почасовой ряд энергопотребления из набора Hourly Energy Consumption. Он подходит для демонстрации тренда, суточной и недельной сезонности и прогнозирования нагрузки.

2. Почему окно равно 168 часам?
168 часов - одна неделя. Такой lookback позволяет модели видеть и суточный, и недельный цикл; выбор дополнительно мотивирован autocorrelation.

3. Почему нельзя случайно перемешать весь временной ряд перед split?
Случайный split переносит будущие наблюдения в train и создаёт утечку. Поэтому используются первые 70% для train, следующие 15% для validation и последние 15% для test.

4. Почему scaler fit выполняется только на train?
Если вычислять mean/std по validation или test, информация о будущем распределении попадёт в preprocessing и исказит независимую оценку.

5. Зачем sin/cos для календарных признаков?
Час 23 и час 0 должны быть близкими значениями. Циклическое кодирование сохраняет такую геометрию для часа, дня недели и дня года.

6. Чем LSTM отличается от GRU?
Обе сети предназначены для последовательностей. LSTM имеет отдельное cell state и больше gates; GRU проще и обычно содержит меньше параметров. В работе выбор между ними делается по validation loss.

7. Почему HuberLoss?
Huber сочетает квадратичную ошибку около нуля и более мягкий линейный штраф для крупных ошибок, поэтому менее чувствителен к редким большим выбросам нагрузки, чем MSE.

8. Зачем нужны baseline?
Нейросеть должна быть полезнее простого правила. Persistence предсказывает предыдущий час, weekly baseline - значение ровно неделю назад. Для энергопотребления второй baseline особенно силён.

9. Чем отличаются MAE, RMSE, R^2 и sMAPE?
MAE даёт среднюю абсолютную ошибку в MW; RMSE сильнее штрафует большие ошибки; R^2 показывает долю объяснённой вариации; sMAPE даёт симметричную относительную ошибку в процентах.

10. Какие фактические метрики получены?
В приложенном состоянии notebook фактические метрики отсутствуют: все code cells не выполнены. Перед защитой нужно запустить notebook и отвечать по сохранённой таблице test_results.

Что обязательно исправить или подготовить до сдачи

- Полностью выполнить notebook и сохранить outputs.
- Подтвердить фактическую загрузку/очистку AEP_hourly.csv.
- Сохранить таблицу hyperparameter search и выбранную LSTM/GRU конфигурацию.
- Сохранить training curves и best checkpoint.
- Получить реальные test MAE, RMSE, R^2 и sMAPE для RNN и baseline.
- Сохранить forecast/residual plots и top-10 ошибок.
- Добавить отдельный финальный раздел «Вывод».

Решение по приёмке

Обязательно исправить до сдачи

- Запустить notebook полностью и сохранить результаты выполнения.
- Предъявить реальные validation результаты hyperparameter search.
- Предъявить реальные test-метрики и сравнение с обоими baseline.
- Сохранить обязательные графики обучения, прогноза и residual analysis.
- Добавить итоговый вывод на основе фактически полученных чисел.

Желательно исправить

- Убрать потенциальную утечку при interpolation до временного split или доказать, что граничные точки не затронуты.
- Сохранять feature_scaler и target_scaler вместе с модельными артефактами.
- Сделать hyperparameter trial сопоставимее: одинаковый seed/пересоздание DataLoader либо несколько повторов.
- Сделать ARTIFACT_DIR переносимым между Colab и локальным Jupyter.
- Добавить установку/проверку версий ключевых зависимостей.

Можно объяснить как ограничение работы

- Всего три hyperparameter trial из-за вычислительных ограничений.
- Прогноз только на один час вперёд, без multi-step horizon.
- Используется только один регион AEP.
- Нет экзогенных признаков погоды, праздников и тарифов.
- Выбор lookback=168 сделан под выраженную недельную сезонность.

Требуется уточнение у преподавателя

- Нужно ли прикладывать best_energy_rnn.pt отдельно от notebook?
- Требуется ли сохранять scaler artifacts для отдельного inference?
- Достаточно ли трёх trial LSTM/GRU для задания по подбору параметров?
- Допускается ли использовать KaggleHub без отдельного Kaggle API token в Colab?
- Нужно ли сравнение с классической моделью ARIMA/Prophet дополнительно к persistence/weekly baseline?